

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

In the modern business landscape, procurement is often a bottleneck. Companies typically juggle multiple vendor quotes that are frequently sent as static PDFs in order to ensure they are getting the best value. However, manual comparison is not just slow; it is prone to human error, often overlooking critical “Red Flags” or failing to account for broader market trends.

Our ProcureIQ: AI-Powered Vendor and Procurement Agent transforms this fragmented process into a seamless, end-to-end automated workflow. By leveraging the advanced General Language Model (GLM) developed by [Z.AI](#), the agent handles everything from data extraction to final outreach.

1.1. Purpose:

The system analysis document highlights the technical scope, data integration strategies and the logic-driven decision-making engine behind the ProcureIQ development. It outlines how the system transitions from unstructured PDF data to automated procurement execution.

So, the key element of system that has been covered are as followed:

1. Architecture:

- a. *The system is built as an AI-orchestrated pipeline using a modular service design. It utilizes [Z.AI](#)'s GLM for natural language understanding and extraction, integrated with a web-scraping module for adversarial research. The infrastructure is designed to be database-centric, ensuring that every automated insight is logged and traceable for audit purposes.*

2. Data Flows:

- a. *Data begins as an unstructured PDF upload from the user. It flows through the Extraction Engine (powered by [Z.AI](#)), where it is transformed into structured JSON data (prices, dates, names). This data then triggers an asynchronous Web Research Flow that fetches external news. All data points, internal quotes and external “Red Flags” will converge in the Central Procurement Database before being served to the Comparison Dashboard.*

3. Model Process:

The model follows a linear automation workflow with a human-in-the-loop validation:

- a. *Ingestion: Parsing vendor quotes into quantifiable metrics.*
- b. *Enrichment: Running adversarial searches to identify vendor risks.*
- c. *Synthesis: Applying “The Brain’s” 4-factor logic (Quotes, News, Benchmarks, History) to generate a recommendation.*
- d. *Execution: Generating a legally formatted Purchase Order (PO) and confirmation draft upon user selection.*

4. Role of Reference:

- a. *This document serves as the primary technical blueprint for Backend Developers, AI Engineers and Procurement Stakeholders. It ensures that the logic used by the “Brain”*

remains consistent and provides a clear map for testers to verify the accuracy of the PDF extraction and the reliability of the automated PO generation.

1.2. Background:

Current procurement workflows are heavily burdened by administrative overhead. While organizations often have established relationships with multiple suppliers, the actual process of gathering quotes and comparing terms remains a fragmented, manual task. Procurement teams are forced to sort through disparate PDF documents, often missing critical market trends or external vendor risks due to the sheer volume of information.

1. Previous Version:

- a. The existing procurement process relies on basic Legacy Database Systems and Manual Entry. In this traditional setup, staff manually input data from emailed quotes into a central spreadsheet or ERP system. While this provides a basic record of transactions, the process lacks any automated intelligence. There is no real-time link between a vendor's quote and their current market reputation or historical pricing performance, leading to a siloed decision-making process where only price but not risk or context is considered.*

2. Changes in Major Architectural Components

The transition to the ProcureIQ introduces several sophisticated components that shift the system from a passive record-keeper to an active decision-support tool:

- a. Automated Ingestion Layer (PDF Parser and GLM): Replacing manual data entry, this component uses pdfplumber to extract text from PDFs, then passes it to Z.AI's GLM to autonomously parse unstructured text into structured JSON fields. No OCR is used — the system handles digital-born or machine-printed PDFs only.*
- b. Adversarial Research Engine: A new, outbound integration that connects the internal database to the live web. It performs real-time scraping to identify "Red Flags" that were previously invisible to the system.*
- c. Multi-Factor Analytical "Brain": This component introduces a logic layer capable of cross-referencing four distinct data streams; current quote data, external research, market benchmarks and historical internal data.*
- d. Automated Fulfillment Module: A shift from manual document drafting to Generative PO Creation. This component automatically synthesizes agreed-upon terms into professional legal documents and email drafts, ready for a final human "one-click" approval.*

3. New capabilities that are only introduced in this

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Procurement Officers	Uploads vendor PDFs, reviews the AI-generated Comparison Dashboard and makes the final vendor selection.	A streamlined interface that highlights risks (Red Flags) and price gaps without manual data entry.
Finance/Management	Approves the final Purchase Order (PO) and monitors budget compliance based on "The Brain's" market benchmarks.	Accurate financial data, cost savings compared to market averages and professional, error-free PO documents.
Vendors	Provide quotes/tenders via PDF and receive automated confirmation emails and POs once selected.	Clear communication regarding terms and faster "Lead-to-PO" cycles for their sales teams.
Development Team	Build and maintain the PDF extraction logic (Z.AI) , the web-scraping "Adversarial" engine and the analytical "Brain".	Clean integration between the GLM and the database, modular search scripts and scalable data processing.
QA Team	Validate the accuracy of the AI's data extraction and ensure the "Red Flag" logic does not return false positives.	High precision in OCR/Extraction, reliable search result parsing and consistent generation of PO documents.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web-based Procurement Portal (Dashboard)
Architecture	AI-orchestrated Pipeline (Modular Services)

The ProcureIQ is structured as an intelligent processing pipeline that connects procurement officers with external vendor data. The system consists of a Web-based Comparison Dashboard that serves as the primary interface for users to upload documents and review AI insights. The backend is composed of four primary modular services:

- 1. Ingestion Service: Powered by [Z.AI](#) GLM to extract structured data from PDF uploads.*
- 2. Research and Risk Service: An adversarial engine that executes web searches to identify vendor “Red Flags”.*
- 3. Analytical “Brain” Service: A logic layer that correlates internal quotes, historical data and market benchmarks.*
- 4. Fulfillment Service: A generative module that drafts formal Purchase Orders and confirmation emails.*

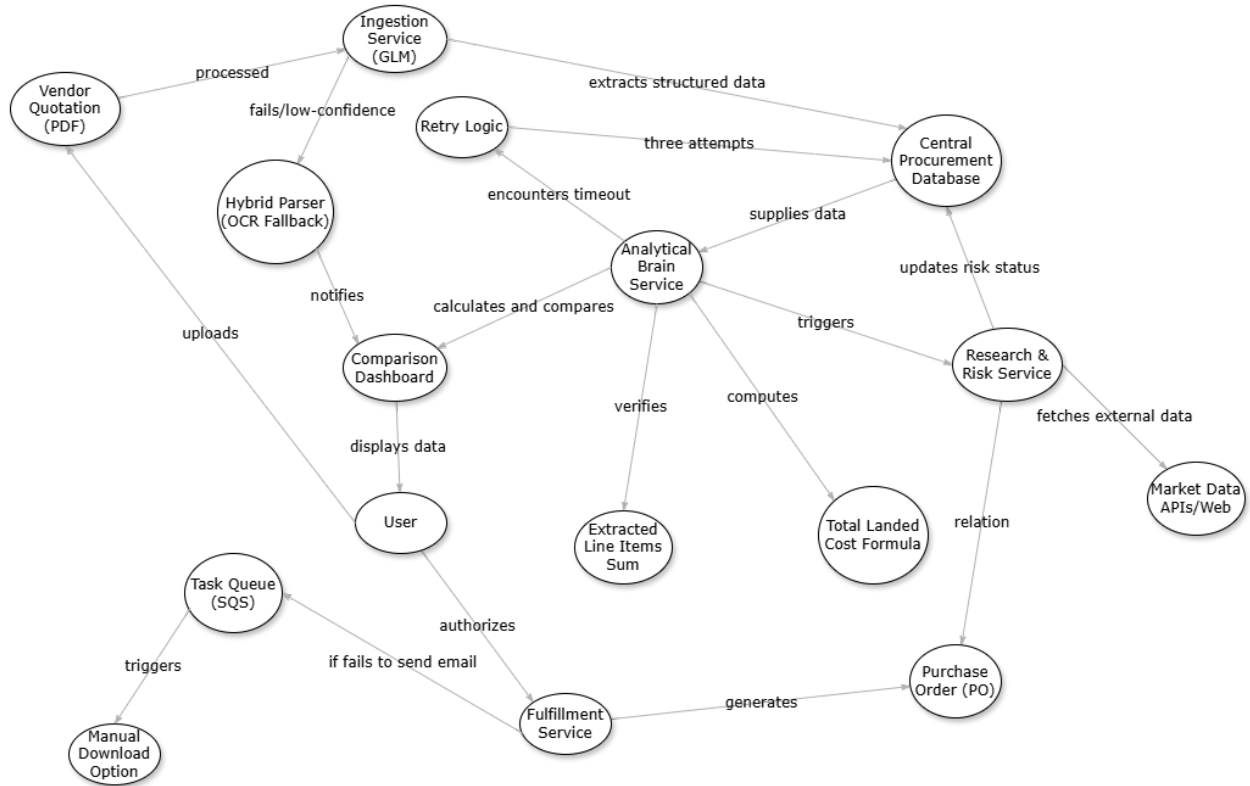
All services are integrated via central Procurement Database, which ensures data persistence across the evaluation lifecycle. The system is deployed on a cloud environment to allow for scalable document processing and real-time web scraping capabilities.

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic “AI Module”. The following Dependency diagram includes:

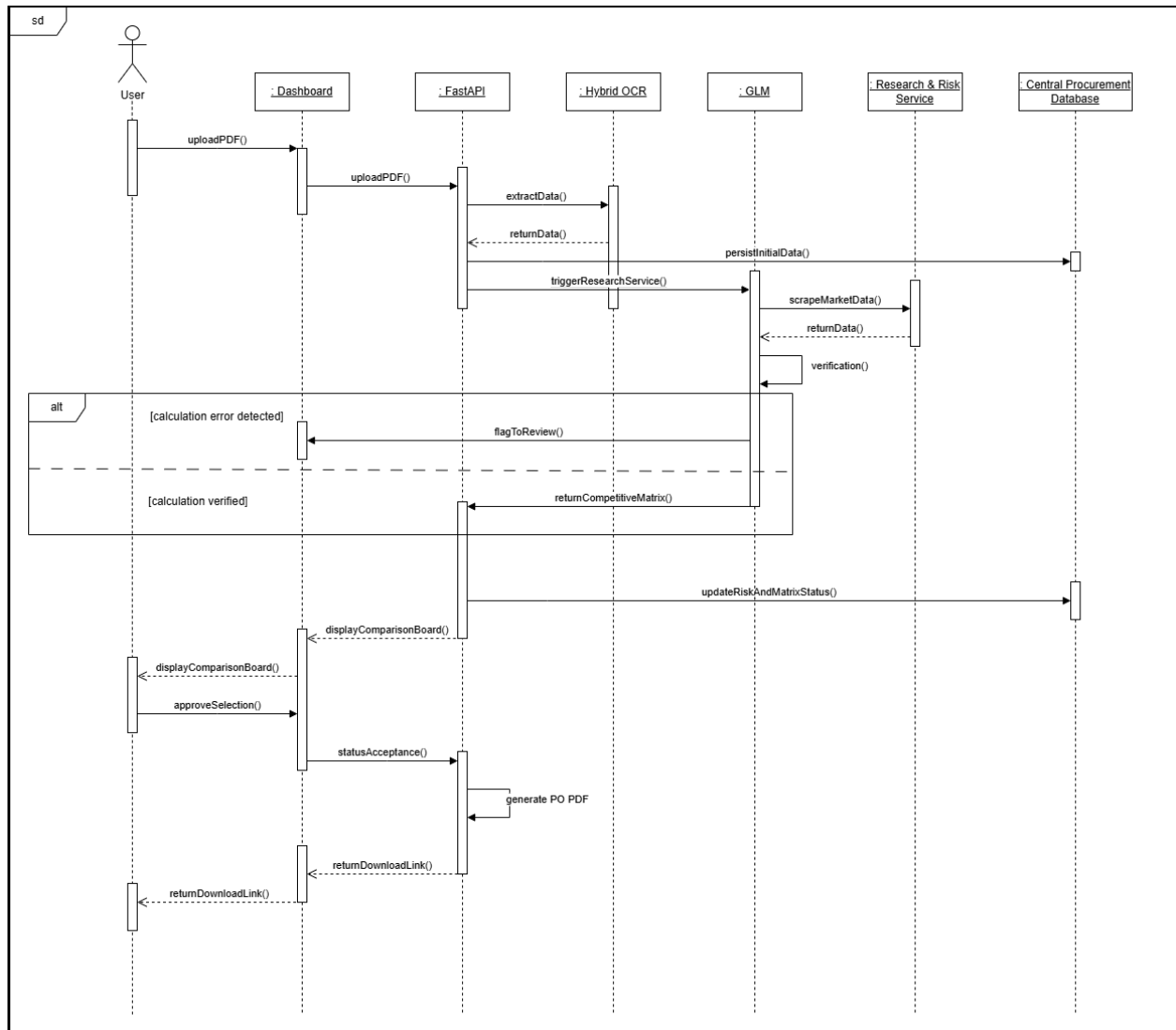
2.1.3 Dependency Diagram

- a. How the prompts are being constructed and sent to the GLM API*
- b. What goes into the context window?*
- c. How is your system receiving, parsing the responses and passing to the next system component?*
- d. Where the limitation of token are enforced or input are chunked before it actually reaches to GLM*
- e. Also, do outline all the major API calls between the system components [e.g. Customer Mobile App calls “Amazon API Gateway” for Order Service]*
- f. Do show how frontend, backend, services, database and external API interacts with each other [e.g. Order service communicates with Amazon RDS to ensure persistence of order data]*
- g. Must highlights the distributed components if the system is planning to be deployed across multiple services*



2.2.2. Sequence Diagram

- Here, do walk through one user interaction flow or customer journey with the stakeholders [e.g. Place Order Flow]
- Show the interactions step by step.
- Do demonstration of each key system features does operate from start to finish



2.2. Technological Stack

2.2.1. Frontend

Streamlit (Python) - Provides a rapid, interactive web dashboard for Procurement Officers. Supports step-by-step workflow (Upload → Review → Research → Analysis → Close Deal) with side-by-side vendor comparison columns, real-time progress indicators, and downloadable PO/email outputs. [e.g. React Native – Cross platform needed for the user base of iOS & Mobile]

2.2.2. Backend

FastAPI (Python) - High performance, asynchronous framework ideal for handling multiple LLM API calls (Z.AI) and background web-scraping tasks without blocking the UI. [e.g. FastAPI – RESTful API for the microservices]

2.2.3. AI Layer

Z.AI GLM-5 API - Utilizes standard chat completion calls with structured system prompts and JSON-in/JSON-out extraction. The GLM handles three tasks: (1) PDF data extraction into

structured JSON via prompt engineering, (2) multi-factor adversarial analysis comparing quotes against market/reputation/history data, and (3) acceptance email drafting. Robust JSON parsing handles code fences, truncated responses, and nested objects.

2.2.4. *Web Research*

DuckDuckGo HTML Scraping (BeautifulSoup) - The system scrapes DuckDuckGo's HTML search results directly using httpx and BeautifulSoup, querying Malaysian marketplaces (Shopee, Lazada) for price benchmarks and vendor names for reputation research. If web scraping yields insufficient pricing data, the system falls back to GLM-estimated Malaysian market prices

2.2.5. *Database*

JSONBin.io - A cloud-based JSON storage service used as the central procurement database. Stores structured vendor quote history (prices, terms, normalized units) with CRUD operations via REST API. Quote history is persisted across sessions and used for historical price comparison in the analytical "Brain" service.

2.2.6. *Cloud/Infra*

Local Deployment (FastAPI + Streamlit) - The system runs locally using a Python launcher (run.py) that starts a FastAPI backend (uvicorn on port 8000) and a Streamlit frontend (port 8501) simultaneously. Both services run on the host machine with no containerization for the MVP.

2.2.7. *Storage*

Local Filesystem (data/) - Vendor quote PDFs are temporarily stored in data/quotes/ during upload processing, and generated Purchase Order PDFs are saved to data/ with naming convention PO-YYYYMMDD-XXXX_<vendor>.pdf. Files are served to the user via FastAPI's FileResponse endpoint.

2.2.8. *Task Queue*

Asyncio (Python) - Asynchronous research tasks are managed using Python's asyncio.gather with staggered execution (0.5s delays between tasks) to avoid rate limiting. Market price research and vendor reputation research run concurrently within the FastAPI async event loop. The frontend displays a spinner while research is in progress.

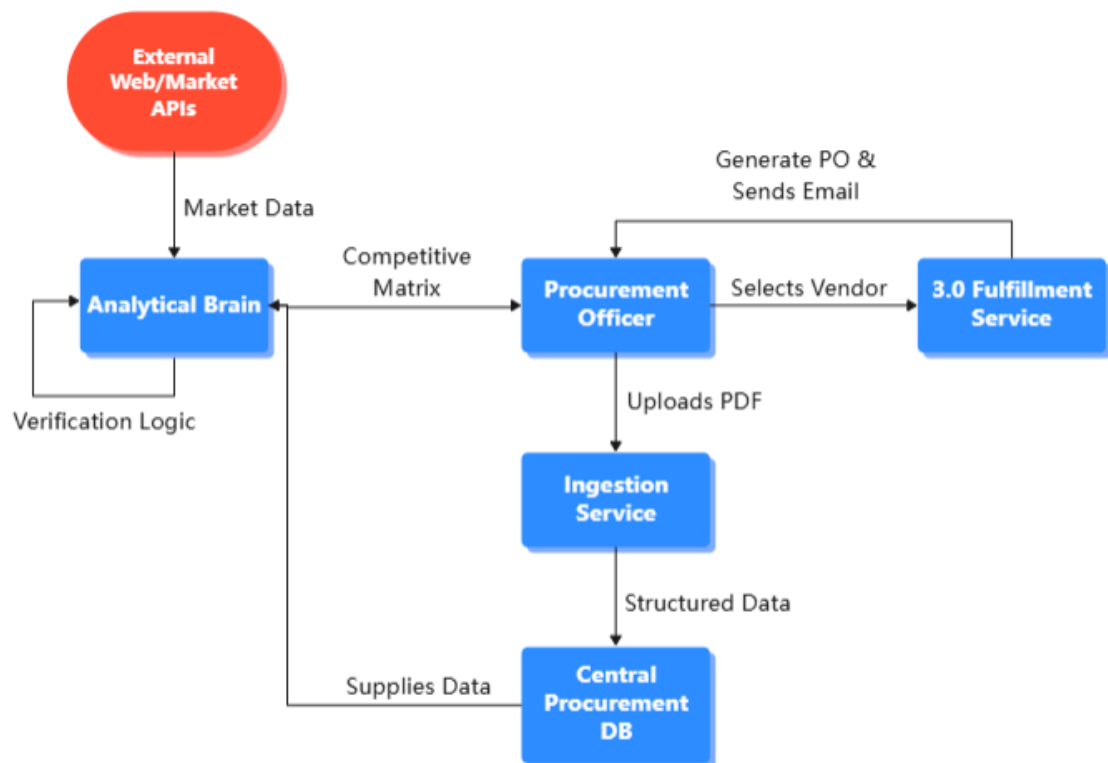
2.3. Key Data Flows

2.3.1. Here, we demonstrate how unstructured PDFs are transformed into structured, actionable intelligence while maintaining high data integrity standards.

2.3.1.1. Data Flow Diagram (DFD)

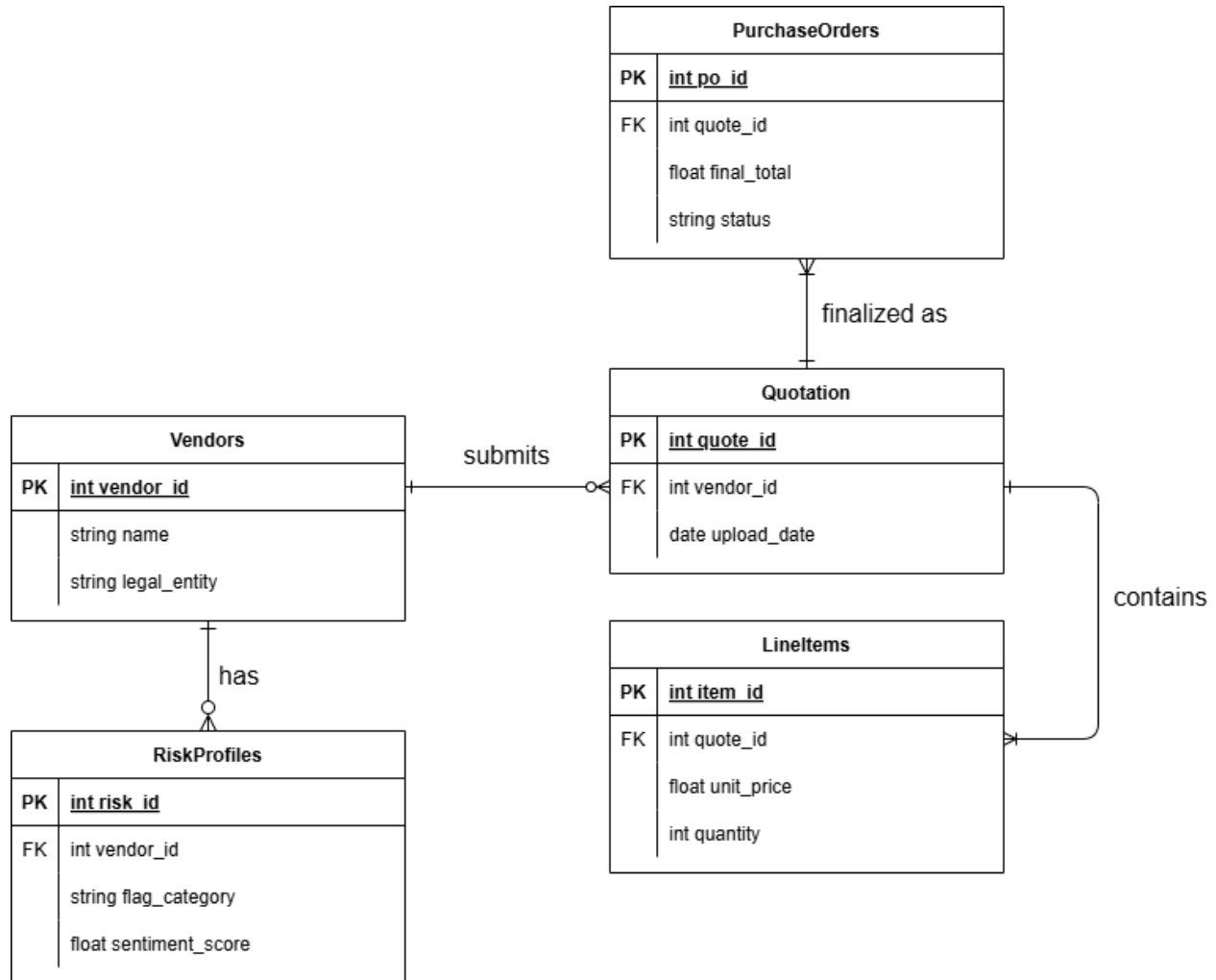
The DFD illustrates how information moves from the Procurement Officer through the AI Layer and into the final Purchase Order (PO)

- Illustrate the data movement across the entire system that includes external entities (users), processes, external service and databases
- Show how proper laws are met (e.g. Black Hole)



2.3.2. Normalized Database Schema

- Show the logical data model (ERD) with Database designed to 3NF form to ensure data integrity and relationships. [e.g. order items is a junction table that is resolving the M: N relationship between orders and menu items.



3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. [e.g. in Table]

#	Features	Description
1	Multi-Format PDF Extraction	The system must ingest vendor quotes in unstructured PDF formats and extract a minimum set of data points (Vendor Name, Unit Price, Tax, Currency, Delivery Date and Terms) with at least 95% accuracy using the Z.AI GLM.

2	<i>Autonomous “Red Flag” Discovery</i>	<i>The system must execute a web-search query for very extracted vendor name to identify “Red Flags” (e.g., bankruptcy or negative social sentiment) and Red flags and positive notes are identified by the GLM based on scraped search results, or via GLM knowledge-based assessment if no web results are found.</i>
3	<i>Multi-Factor Comparison Logic</i>	<i>The system must generate a recommendation based on a weighted analysis of:</i> - <i>Cost (against internal history and market benchmarks)</i> - <i>Speed (delivery window)</i> - <i>Risk (external research results)</i>
4	<i>Comparison Dashboard Visualization</i>	<i>The UI must provide a side-by-side view of at least 3 vendors, highlighting the “Best Value” choice and identifying price anomalies (e.g., a vendor charging 20% above the market average)</i>
5	<i>Automated PO and Communication Generation</i>	<i>Upon selection, the system must generate a finalized PDF Purchase Order and a draft acceptance email that automatically pulls the specific price and terms agreed upon in the original quote.</i>

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g.)
Scalability	<i>The system must handle batch uploads (e.g., 20+ quotes at once) without timing out during peak procurement cycles.</i>	<i>Use AWS Lambda for document parsing and SQS (Simple Queue Service) to decouple the PDF ingestion from the UI, ensuring no files are lost.</i>
Accuracy (Precision)	<i>Data extraction from vendor PDFs must maintain a 95% confidence score for financial fields (Price, Total).</i>	<i>Use Z.AI GLM with structured JSON output via prompt engineering and a robust JSON extraction parser that handles code fences, truncated responses, and nested objects.</i>

		A confidence score (0.0–1.0) is returned for each extraction.
<i>Data Security</i>	<i>Vendor quotes and Purchase Orders contain sensitive pricing; data must be encrypted at rest and in transit.</i>	API keys and configuration are stored locally in a .env file (excluded from version control). All external API calls use HTTPS/TLS. JSONBin.io access is authenticated via a master key.
<i>Token Latency</i>	<i>The “Analysis Brain” must deliver a vendor recommendation within 10 seconds for up to 5 vendor comparisons.</i>	Implement Async processing for LLM calls. The frontend displays a “Reasoning...” status bar to manage user expectations during deep analysis.
<i>Cost Efficiency</i>	<i>Minimize LLM costs by avoiding redundant processing of the same vendor PDFs.</i>	Each processed quote generates a raw_text_hash stored in JSONBin. While the hash is recorded for future deduplication, the MVP currently re-processes uploads. Text input to GLM is truncated to 4000 characters (with a 1500-character fallback) to manage token usage.
<i>Auditability</i>	<i>Every AI-generated decision must be traceable back to the source data for compliance audits.</i>	All quote history records are persisted in JSONBin.io with timestamps, raw text hashes, and extraction results. Every AI-generated decision (analysis scores, red flags) can be traced back to the source quote data in the session.

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for “Grand Vision” but not part of this UMHackathon 2026. [e.g. below]

- Multi-ERP Integration: Deep bidirectional sync with SAP or Oracle (MVP will use manual export/import).*
- Negotiation Agent: AI-to-AI price negotiation (MVP is focused on selection, not haggling).*

- c. *OCR for Handwritten Quotes: Only digital or scanned machine-printed PDFs are supported.*
- d. *Full Contract Lifecycle Management (CLM): Managing long-term legal amendments after the PO is sent.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

- 4.1.1. **Grayscale Rollout & A/B Testing:** *The deployment follows a Canary Release strategy to mitigate the risk of incorrect data extraction or faulty “Brain” recommendations. The AI extraction and Adversarial Research features are initially released to 5% of the Procurement Team (Beta group). We monitor the “Human Correction Rate” that is how often users manually override the AI’s extracted unit prices. If the error rate stays below 5% over a 2-day period, the rollout will then be gradually increased until 100%. After it passes, A/B testing is conducted on the “Brain’s” recommendation engine, which variants of two vendors are better. (e.g., variant A prioritizes cost whereas variant B prioritizes reliability).*
- 4.1.2. **Emergency Rollback (ER) & Golden Release:** *In procurement, a system failure can halt the entire supply chain, making recovery strategies critical. If the GLM starts producing “hallucinated” data (e.g., RM10,000 instead of RM100) or if the latency for web research exceeds 60 seconds, these are considered as anomalies, an automated ER is triggered. The AWS CodePipeline automatically reverts the service to the Golden Version (the last stable build confirmed to have 99% extraction accuracy). If the external [Z.AI](#) API is down, the system triggers a “Fail-Safe” mode, where the AI features are disabled and the dashboard reverts to a manual input form, ensuring procurement officers can still work without AI assistance.*

4.1.3. Priority Matrix:

4.1.3.1. P1 Critical:

- *If the AI-extracted total price differs from the sum of line items in more than 2% of uploads, emergency rollback to the Golden version.*
- *If the dashboard or [Z.AI](#) API integration is completely inaccessible, immediate SMS/PagerDuty notification will be sent to the DevOps and AI engineering teams.*

4.1.3.2. P2 High:

- *If the Adversarial Research engine fails to return results for more than 20% of new vendors, the Red Flag automated feature will be temporarily disabled and notify users that research must be done manually.*

- If analysis takes more than 60 seconds per quote, scale AWS Lambda/ECS instances.

4.1.3.3. P3 Medium:

- If the market benchmark data has not been updated for over 30 days, trigger a manual refresh of the benchmarking database.
- If AI misses non-financial terms (e.g., delivery notes) but keeps prices correct, queue the specific PDFs for the development team to refine the extraction prompts.

4.1.3.4. P4 Low:

- If misalignment of columns in the comparison dashboard happens on certain mobile browsers, add to the next sprint for UI/UX refinement.
- If users give feedback that the PO email draft is “too informal”, adjust the system prompt for the fulfillment service in the next scheduled deployment.
- Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project. [e.g. P1 Critical: If payment failure error is more than 1%, emergency rollback to Golden release.]

4.2. Monitoring:

4.2.1. *The Three Layers of Health: We monitor the system across three distinct planes to ensure that a failure in one (like an AI hallucination) does not crash the entire procurement process.*

- For infrastructure, the key Service Level Indicators (SLIs) are API Uptime, database latency, S3 bucket accessibility. Tools used are AWS CloudWatch and Datadog.
- For AI intelligence, the key SLIs are extraction confidence scores, hallucination rate and token usage/cost. Tools used are LangSmith and custom log traces.
- For data integrity, the key SLIs are PO Math Accuracy (unit * qty) and schema validation failures. The tools used are PostgreSQL constraints and great expectations.

4.2.2. *Automated Trigger and Action Logic: If the monitoring system detects damage to the system’s health, it triggers predefined responses based on the severity of the breach.*

- The system uses a confidence score (0.0–1.0) returned by the GLM for each extracted item. Low confidence scores flag extractions for manual review. The extracted data is displayed to the user in Step 2 (Review Extracted Data) with the confidence bar visible, allowing human-in-the-loop verification before proceeding.
- The GLM extraction prompt instructs the model to extract the total item price as-is from the PDF. The system calculates $\text{price_per_normalized_unit} = (\text{item_price} / \text{normalized_quantity})$ for display and comparison purposes. Full math cross-validation ($\text{unit price} \times \text{qty} = \text{total}$) is identified as a future enhancement.
- The web scraping service uses an httpx timeout of 15 seconds per request. If a search request times out, the system gracefully falls back to GLM knowledge-based assessment for market prices or vendor reputation, ensuring the user still receives results even when web scraping is unavailable.

4.2.3. *Monitoring Dashboard (Internal POV): The development team uses a centralized dashboard to track the “Health Score” of the AI agent in real-time.*

- *Human-in-the-loop (HITL) Rate that shows how many AI recommendations were overridden by the procurement officer.*
- *Semantic similarity that compares the Red Flags found today against historical flags to ensure the web-crawler has not been blocked by new sites.*
- *Budget burn that tracks real-time [Z.AI](#) token costs to ensure the “Cost-per-Quote” remains within the defined budget.*

4.2.4. *Summary of System Health Triggers*

- *The system operates normally and auto-generates PO drafts if the accuracy is more than 95% and latency of less than 5 seconds; indicates that the system health is healthy (green).*
- *System disable auto-drafting and force users to manually click “Verify” on every field when the accuracy is ranging from 90% to 95% with high latency; indicates that the system health degraded (yellow).*
- *System revert to the Golden Release version if the accuracy is less than 90% or math errors; indicates that the system health is critical (red).*
- *Show the planning for your monitoring plan of the system and how it will trigger if any damage is detected of the health of the system.*

4.3. **Assumptions:** *State key operational & environmental conditions that you assumed to be valid while development of system and deployment. [e.g.*

- Document standard and quality: It is assumed that 95% of vendor quotes are provided in digital-born PDF formats (not hand-written or extremely low-quality physical scans) and are written in English or supported regional languages for accurate GLM extraction.*
- Internet connectivity and latency: Procurement officers are assumed to have a stable, high speed corporate internet connection. This is critical because the system must upload large PDF files and maintain a persistent connection for real-time web-scraping Adversarial Research.*
- Cloud service availability: The system assumes that [Z.AI](#) and web-search APIs maintain their service-level agreements, as the “Brain” and “Red Flag” features are entirely dependent on their uptime.*
- Database baseline: It is assumed that historical procurement data and market benchmarks are already present in the database or will be provided in a structured format (CSV/Excel) to allow the AI to perform meaningful price comparisons.*

- e. *Vendor communication standards: It is assumed that the external vendors can receive and process Purchase Orders (POs) and confirmation emails sent via standard SMTP protocols.*
- f. *User technical literacy: It is assumed that the primary users (Procurement Officers) are familiar with digital dashboards and understand that AI-generated suggestions require a final "Human-in-the-loop" verification before financial commitment.*
- g. *Legal compliance: It is assumed that the company has the legal right to store vendor quote data and perform public-domain web research on these vendors for the purpose of risk mitigation.*

4.4. **External Dependencies:** Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks
Z.AI GLM API	The core reasoning engine. Handles PDF data extraction, multi-factor analysis and generates structured JSON output for the dashboard.	Critical - Rate limits or API outages stop all automation. Mitigation: retry logic with exponential backoff (3 attempts).
JSONBin.io	Cloud JSON storage acting as the central procurement database for quote history persistence.	High - If JSONBin is inaccessible, historical price comparisons are unavailable. Mitigation: in-memory session data still works for current analysis.
DuckDuckGo (HTML Scraping)	Conducts market price research and vendor reputation searches by scraping search results.	Medium - If DuckDuckGo blocks requests or changes HTML structure, web research fails. Mitigation: GLM knowledge-based fallback for both pricing and reputation.
pdfplumber	Extracts text from uploaded vendor PDF quote documents.	Low - May fail on scanned/image-only PDFs. Mitigation: system returns confidence 0.0 and flags for manual review.

5. Project Management & Team Contributions

© UMHackathon 2026

Email: umhackathon@um.edu.my

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. **Project Timeline:** *Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]*

- Day 1-2: Project Planning and Role Assignment
- Day 3: Finalize project scope
- Day 4: Meeting with Mentor
- Day 5: Z AI Workshop
- Day 6: Complete System Analysis Documentation (SAD)
- Day 7: Got API Token and start coding for backend
- Day 8: Start coding for frontend and complete the other two documentations
- Day 9: Record and finalize pitching video
- Day 10: Submit documentations, coding notebook and pitching video

5.2. **Team Members Role:**

1. Fatnin - AI Architect - Prompt engineering and agentic logic
2. Iman - Backend and Tooling Engineer - APIs and external integrations
3. Amira - Frontend Developer (UI/UX) - Design UI website
4. Husna - Integration and Data Specialist - Database and consistency
5. Syakirin - Product Manager and Pitcher - Storytelling and business value

5.3. **Recommendations:** *If any recommendation is applicable in terms of scalability, performance and reliability [e.g. using Redis Cache for Menu data]*

- *Reliability: Instead of relying on a single extraction method, implement Hybrid Parsing (Lattice + Stream). The system should score the output of both; if the confidence score is low, trigger an OCR-backed fallback to ensure tables are captured accurately.*
- *Performance: Use RedisVL to implement a semantic cache. This avoids redundant calls to the Z.AI GLM by storing and retrieving semantically similar search results or PDF summaries, reducing latency by up to 40%.*
- *Scalability: Implement a vector store (e.g., pgvector in Postgres or Qdrant) to store embeddings of all previous procurement cycles. This allows "The Brain" to perform real-time Retrieval-Augmented Generation (RAG) to compare current quotes against years of historical pricing.*
- *Cost Efficiency: Use a cheaper model for basic text cleaning and metadata extraction, and reserve the high-power [Z.AI](#) GLM only for the final "Multi-Factor Analysis" and decision-making step.*